Monash

iOS App Development

OnTrack Submission

Wow Factor Proposal

Submitted By:
Aryan Cyrus
amaj0006
2026/09/19 23:31

Tutor:
Tanzim Khan

September 19, 2026



Wow Factor Proposal

Name: Aryan Cyrus
Student ID: 33114242
Date: 15/09/2026

For this proposal, you will need to write as much as needed for your demonstrator to approve your concept **(Some sections might not be required**). Ensure you seek their initial approval before writing this document.

Learning Journey and Evidence

The purpose of the feature

To Train a small CoreML regression/classifier model that predicts a project's likely cost range from its attributes (project type, floor area, permit type).

The problem or user need it addresses

Users often need cost estimates for their projects. This will address that need by using a trained model to output an estimated project cost based on historical data from the Victorian Building and Plumbing Commission.

How it improves your existing application

In my app proposal, a key demonstrator critique read as follows:
“you have an immense amount of integration risk spread across multiple external services. In particular, accessing an official Bunnings commercial API for material costs requires formal enterprise partner agreements or complex OAuth2 sandbox workflows”

This model directly replaces the Bunnings supplier API. The reason this needs to be replaced is hinted at above; “accessing an official Bunnings commercial API for material costs requires formal enterprise partner agreements or complex OAuth2 sandbox workflows”.

That's a major gap in my app; I have taken this as an opportunity to create a ‘Wow Factor’.

Why it provides a unique or memorable user experience

Most information in GroundTruth (planning overlays, permit requirements, AI chat answers) is retrieved or generated from an external source. The cost estimate is the one feature that runs entirely offline. It is to be a real dollar figure that appears instantly, with no loading spinner waiting on a third-party API, because inference happens fully on-device via CoreML. This makes it feel fast and dependable in a way the rest of the app cannot. It also addresses a very common painpoint; cost is one of, if not the most important aspect of a project. Having a reliable local solution provides real value to the user.

The iOS frameworks, APIs, or technologies required

Create ML	Used to train the model from cleaned permit data; a build-time tool, not shipped in the app itself
CoreML	loads the trained model and runs on-device inference at runtime.
SwiftUI	Used to visualise the data/cost in UI elements
SwiftData	Used to persist data for a given project

The major components that will need to be developed

1) An offline data-cleaning and feature-engineering pipeline (Python/pandas) that turns the raw VIC Building Permit Activity Data into a training-ready CSV.
2) The trained CoreML tabular regressor (CostEstimator.mlmodel), bundled into the Xcode project.
3) A SwiftUI “Cost Estimate” view on the Project detail screen. This may also be a section/row on the detailed view, not necessarily a separate view.

Any research required before implementation

1) Downloading and understanding the schema of the BPC/VBA Building Permit Activity Data. I cannot read this without cleaning it and knowing its format.
2) Learning Create ML's Tabular Regressor workflow
3) Understanding how a CoreML tabular model behaves when given a category it was not trained on, to design a fall back in case the figure output is absurd.

Why the feature is challenging to implement

1) It is a feature/tech stack that I have 0 experience with.
2) I will need to learn how to make a full ML pipeline within weeks.
3) The output needs to be somewhat reliable and high quality; I do not yet know how to optimize this.

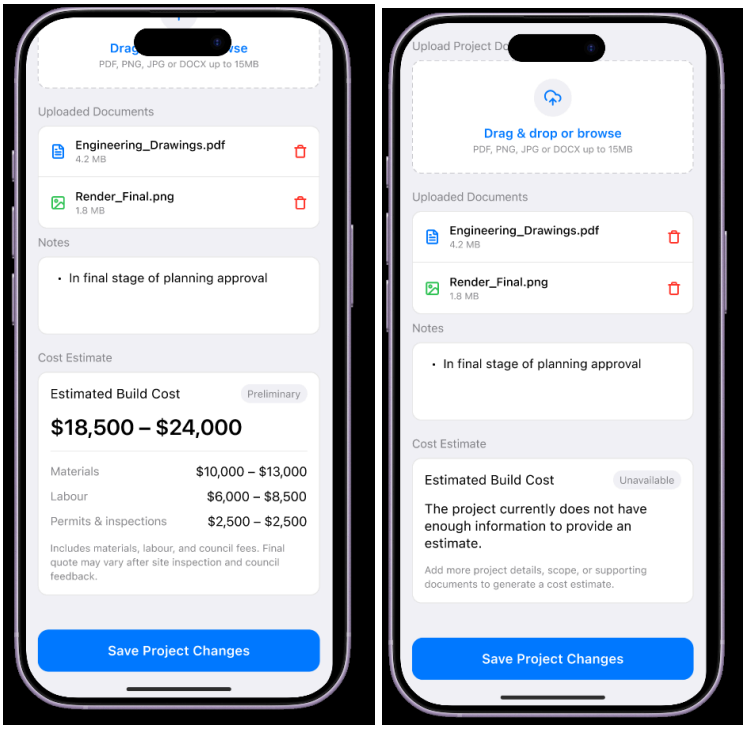
Which concepts from the unit it builds upon (or what outside the unit)

Unit Content	Outside Unit Content
SwiftData persistence and the SwiftUI form/detail-view patterns already used elsewhere in GroundTruth (and practised in earlier labs), plus the general pattern of binding a UI to a data model.	Creating a full ML pipeline and integrating it into my swift project,

Which new concepts or technologies you intend to explore

Refer to the iOS frameworks section above.

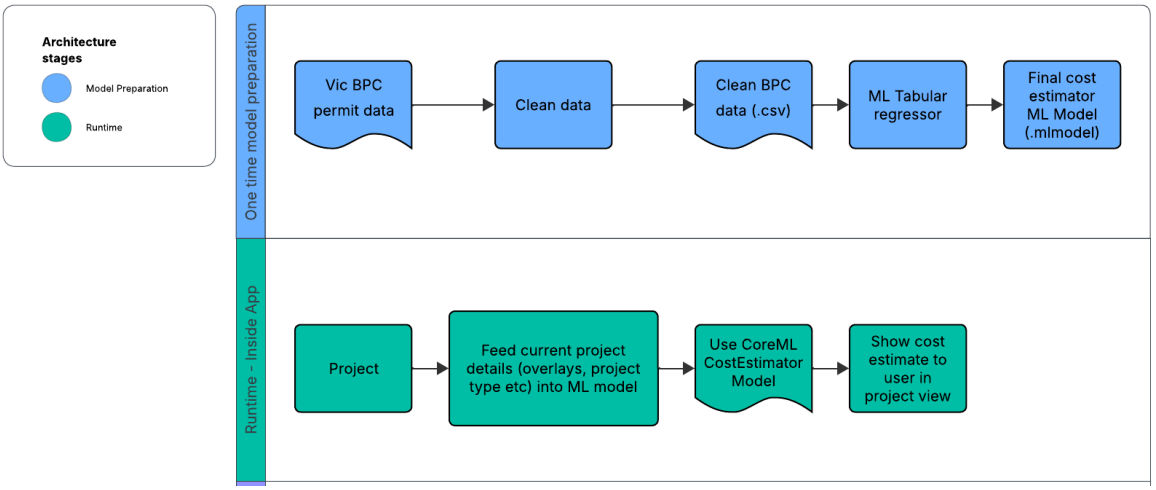
Interface mock-ups



Workflow diagrams

1) User clicks on a given project
2) App checks if there is enough information to provide an estimate (at minimum a project description, and title are needed, but the more information there is the better/more accurate the advice will be)
3) The project view shows either an estimate OR a prompt indicating more information is needed for an estimate.

Architecture diagrams



User interaction sketches

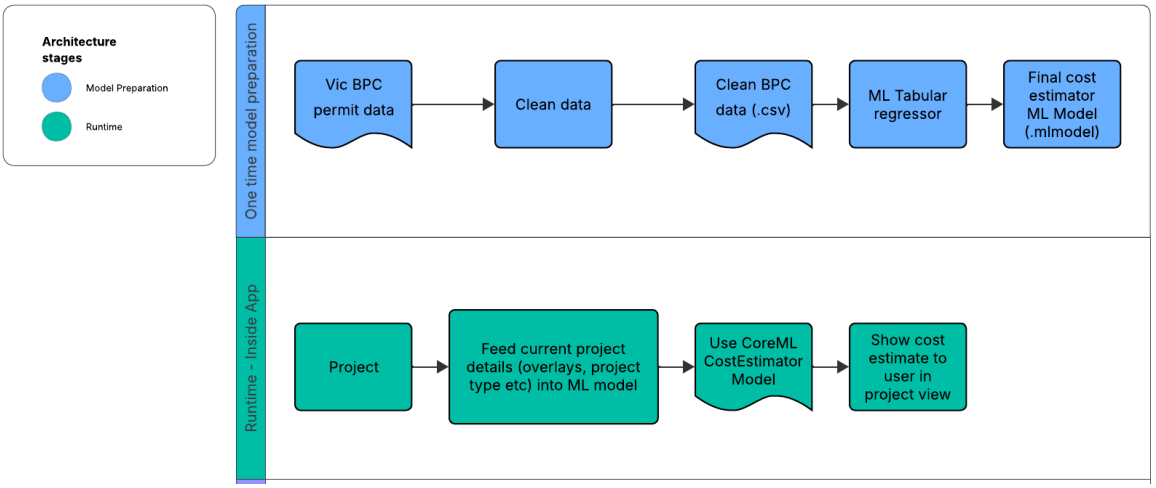
N/A

Other relevant design material

N/A

The order in which the feature will be developed and The expected milestones

As per the order in this architecture diagram:



MVP elements of the feature

- A trained CoreML model that returns a cost prediction for a project.

- A working SwiftUI view displaying that prediction on the Project screen.

Any risks that may affect successful completion

- Real government data may be messier than expected (inconsistent formatting across years), extending the cleaning step.
- The info set is limited (postcode, project type, floor area), so prediction accuracy will be modest.
- New Technology with which I have no experience; this element of unknowns is a risk of its own.
- Limited time.

References

[1] “Core ML | Apple Developer Documentation,” Apple Developer Documentation. https://developer.apple.com/documentation/coreml/

[2] “Building permit activity summaries” Building and Plumbing Commission. Data | Building and Plumbing Commission